MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Gendal Lal

Subject & Subject Code: Theory of Computation & 4CSE4-03, 4AM4-03, 4AD4-03

Branch: CSE, AIML and AI & DS

Batch:

Semester: IV

Session: 2023-24(Even Sem.)

Unit No.: 1

Date of Submission: 07/02/2024

Lect.No.	Topics Covered	Page No.	Total pages
1	Introduction to TOC	1-1	1
2	Finite State Machine	2-4	3
3	Transition Graph	5-5	1
4	DFA & NFA	6-8	3
5	Conversion of NFA into DFA	9-11	3
6	Minimization of DFA	12-13	2
7	Mealy Machine	14-17	4
8	Moore Machine	18-20	3

References Book:

1. "Introduction to the Theory of Computation "by Micheal Sipser
2. "Introduction to computability "by Fred C Henie

G.Lal

FACULTY

HOD

6/12/2023

DEAN ACADEMICS

UNIT-1
Lec-1

What is TOC?

In theoretical computer science, the **theory of computation** is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches: automata theory, computability theory and computational complexity theory.

In order to perform a rigorous study of computation, computer scientists work with a mathematical abstraction of computers called a model of computation. There are several models in use, but the most commonly examined is the Turing machine.

Automata theory

In theoretical computer science, **automata theory** is the study of abstract machines (or more appropriately, abstract 'mathematical' machines or systems) and the computational problems that can be solved using these machines. These abstract machines are called automata.

This automaton consists of

- **states** (represented in the figure by circles),
- and **transitions** (represented by arrows).

As the automaton sees a symbol of input, it makes a *transition* (or *jump*) to another state, according to its ***transition function*** (which takes the current state and the recent symbol as its inputs).

Uses of Automata: compiler design and parsing.

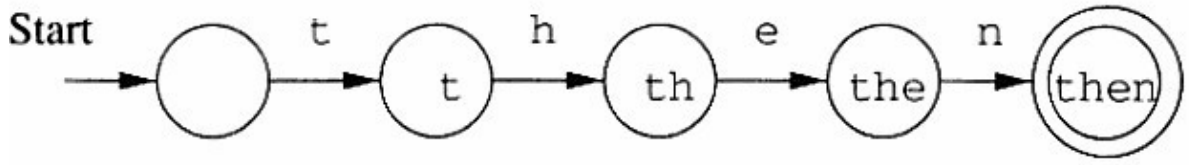


Figure 1.2: A finite automaton modeling recognition of **then**

Finite Automata(FA) is the simplest machine to recognize patterns.The finite automata or finite state machine is an abstract machine which have five elements or tuple. It has a set of states and rules for moving from one state to another but it depends upon the applied input symbol. Basically it is an abstract model of digital computer. Following figure shows some essential features of a general automation.

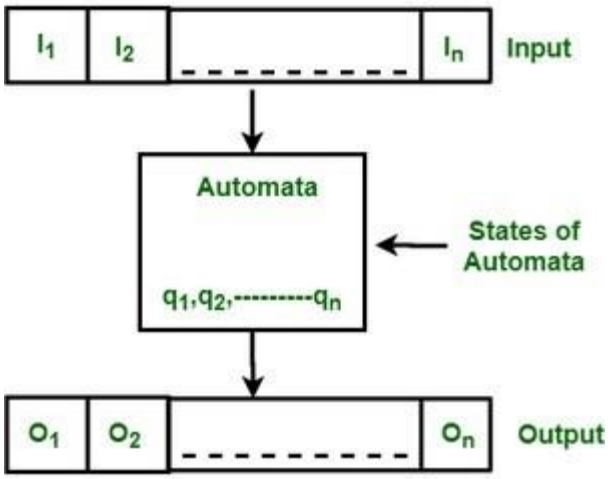


Figure: *Features of Finite Automata*

The above figure shows following features of automata:

1. Input
2. Output
3. States of automata
4. State relation
5. Output relation

A Finite Automata consists of the following :

Q : Finite set of states.

Σ : set of Input Symbols.

q : Initial state.

F : set of Final States.

δ : Transition Function.

Formal specification of machine is
{ Q, Σ, q, F, δ }.

FA is characterized into two types:

1) Deterministic Finite Automata (DFA)

DFA consists of 5 tuples {Q, Σ, q, F, δ}.
Q : set of all states.

Σ : set of input symbols. (Symbols which machine takes as input)

q : Initial state. (Starting state of a machine)

F : set of final state.

δ : Transition Function, defined as δ : Q X Σ --> Q.

In a DFA, for a particular input character, the machine goes to one state only. A transition function is defined on every state for every input symbol. Also in DFA null (or ε) move is not allowed, i.e., DFA cannot change state without any input character.

For example, below DFA with Σ = {0, 1} accepts all strings ending with 0.

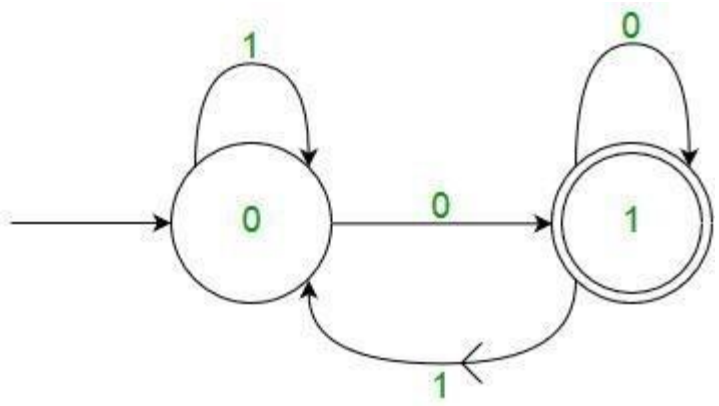


Figure: *DFA with Σ = {0, 1}*

One important thing to note is, ***there can be many possible DFAs for a pattern***. A DFA with minimum number of states is generally preferred.

2) Nondeterministic Finite Automata(NFA)

NFA is similar to DFA except following additional features:

1. Null (or ε) move is allowed i.e., it can move forward without reading symbols.
2. Ability to transmit to any number of states for a particular input.

However, these above features don't add any power to NFA. If we compare both in terms of power, both are equivalent.

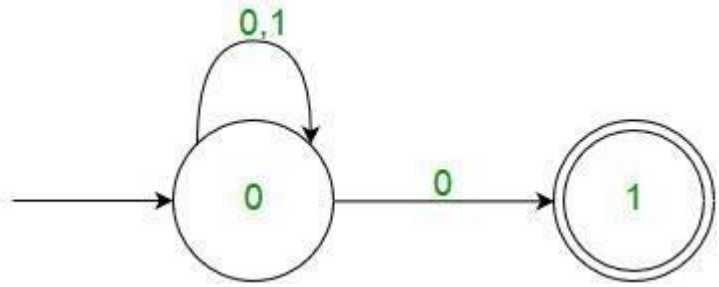
Due to above additional features, NFA has a different transition function, rest is same as DFA.

δ: Transition Function

δ: Q X (Σ U ε)moveisnotallowed,i.e.,DFA) --> 2 ^ Q.

As you can see in transition function is for any input including null (or ε), NFA can go to any state number of states.

For example, below is a NFA for above problem



NFA

One important thing to note is, ***in NFA, if any path for an input string leads to a final state, then the input string accepted***. For example, in above NFA, there are multiple paths for input string "00". Since, one of the paths leads to a final state, "00" is accepted by above NFA.

Some Important Points:

- **Justification:**

Since all the tuples in DFA and NFA are the same except for one of the tuples, which is Transition Function (δ)
In case of DFA

$\delta : Q \times \Sigma \dashrightarrow Q$

In case of NFA

$\delta : Q \times \Sigma \dashrightarrow 2^Q$

Now if you observe you'll find out $Q \times \Sigma \rightarrow Q$ is part of $Q \times \Sigma \rightarrow 2^Q$.
In the RHS side, Q is the subset of 2^Q which indicates Q is contained in 2^Q or Q is a part of 2^Q, however, the reverse isn't true. So mathematically, we can conclude that **every DFA is NFA but not vice-versa**. Yet there is a way to convert an NFA to DFA, so **there exists an equivalent DFA for every NFA**.

1. Both NFA and DFA have same power and each NFA can be translated into a DFA.
2. There can be multiple final states in both DFA and NFA.
3. NFA is more of a theoretical concept.
4. DFA is used in Lexical Analysis in Compiler.

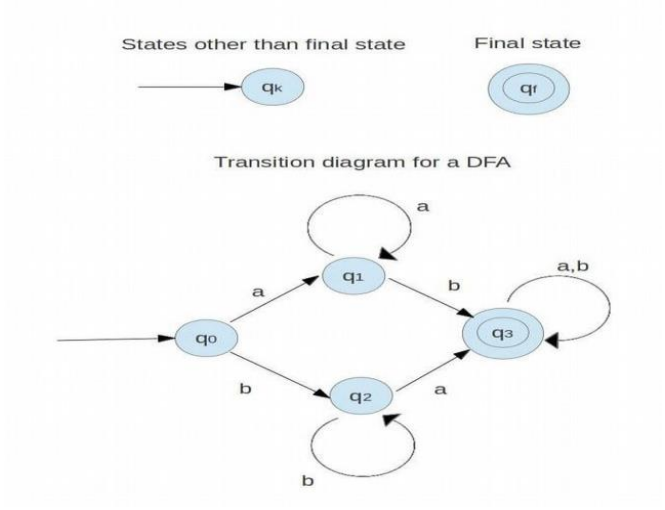
Lec-2

Transition graphs

Transition graph can be interpreted as a flowchart for an algorithm recognizing a language. A transition graph consists of three things:

1. A finite set of states, at least one of which is designated the start state and some of which are designated as final states.
2. An alphabet Σ of possible input symbols from which the input strings are formed.
3. A finite set of transitions that show the change of state from the given state on a given input.

A successful path through the transition graph is a series of edges forming a path beginning at the start state and ending at one of the final states.Some of the transition graphs are given below



The transition graph in the figure represents a DFA with at least one a and at least one b.

A transition diagram for DFA, M = (Q, Σ, δ, F, q_0) is a graph defined as follows:
(a) For each state in Q there is a node represented by the circle.
(b) For each state q in Q each input symbol a in Σ, let δ(q, a) = q'. Then transition diagram has an arc from q to q', labelled a.
(c) There is always an arrow into the start state that is not from any other state.
(d) Nodes corresponding to accepting states are denoted by a double circle. Rest of the states are depicted by a single circle.

Lec-3

1) Deterministic Finite Automata (DFA)

DFA consists of 5 tuples {Q, Σ, q, F, δ}.
Q : set of all states.

Σ : set of input symbols. (Symbols which machine takes as input)

q : Initial state. (Starting state of a machine)

F : set of final state.

δ : Transition Function, defined as δ : Q X Σ --> Q.

In a DFA, for a particular input character, the machine goes to one state only. A transition function is defined on every state for every input symbol. Also in DFA null (or ∈) move is not allowed, i.e., DFA cannot change state without any input character.

For example, below DFA with Σ = {0, 1} accepts all strings ending with 0.

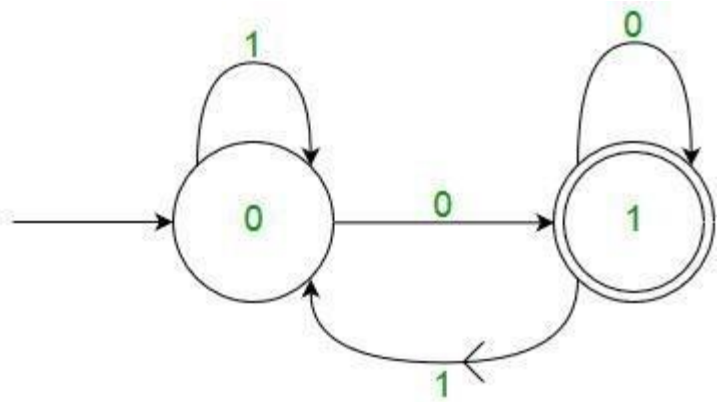


Figure: *DFA with Σ = {0, 1}*

One important thing to note is, ***there can be many possible DFAs for a pattern***. A DFA with minimum number of states is generally preferred.

Lec-4

Nondeterministic Finite Automata(NFA)

NFA is similar to DFA except following additional features:

1. Null (or) move is allowed i.e., it can move forward without reading symbols.
2. Ability to transmit to any number of states for a particular input.

However, these above features don't add any power to NFA. If we compare both in terms of power, both are equivalent.

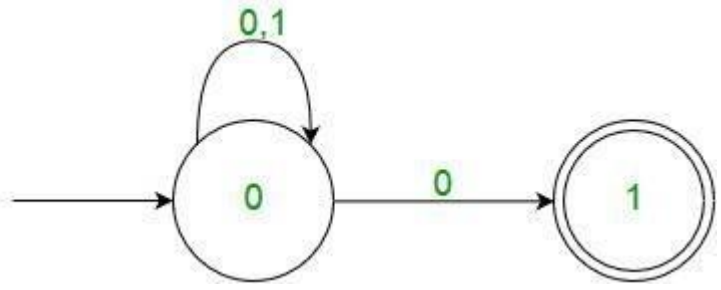
Due to above additional features, NFA has a different transition function, rest is same as DFA.

δ: Transition Function

δ: Q X (Σ U ε)moveisnotallowed,i.e.,DFA) --> 2 ^ Q.

As you can see in transition function is for any input including null (or), NFA can go to any state number of states.

For example, below is a NFA for above problem



NFA

One important thing to note is, ***in NFA, if any path for an input string leads to a final state, then the input string accepted***. For example, in above NFA, there are multiple paths for input string "00". Since, one of the paths leads to a final state, "00" is accepted by above NFA.

Some Important Points:

- **Justification:**

Since all the tuples in DFA and NFA are the same except for one of the tuples, which is Transition Function (δ)
In case of DFA

$\delta : Q \times \Sigma \dashrightarrow Q$

In case of NFA

$\delta : Q \times \Sigma \dashrightarrow 2^Q$

Now if you observe you'll find out $Q \times \Sigma \rightarrow Q$ is part of $Q \times \Sigma \rightarrow 2^Q$.
In the RHS side, Q is the subset of 2^Q which indicates Q is contained in 2^Q or Q is a part of 2^Q, however, the reverse isn't true. So mathematically, we can conclude that every DFA is NFA but not vice-versa. Yet there is a way to convert an NFA to DFA, so there exists an equivalent DFA for every NFA.

2. Both NFA and DFA have same power and each NFA can be translated into a DFA.
2. There can be multiple final states in both DFA and NFA.
3. NFA is more of a theoretical concept.
4. DFA is used in Lexical Analysis in Compiler.

Lec-5

Conversion from NFA to DFA

In this section, we will discuss the method of converting NFA to its equivalent DFA. In NFA, when a specific input is given to the current state, the machine goes to multiple states. It can have zero, one or more than one move on a given input symbol. On the other hand, in DFA, when a specific input is given to the current state, the machine goes to only one state. DFA has only one move on a given input symbol.

Let, M = (Q, ∑, δ, q0, F) is an NFA which accepts the language L(M). There should be equivalent DFA denoted by M' = (Q', ∑', q0', δ', F') such that L(M) = L(M').

Steps for converting NFA to DFA:

Step 1: Initially Q' = ϕ

Step 2: Add q0 of NFA to Q'. Then find the transitions from this start state.

Step 3: In Q', find the possible set of states for each input symbol. If this set of states is not in Q', then add it to Q'.

Step 4: In DFA, the final state will be all the states which contain F(final states of NFA)

Example 1:

Convert the given NFA to DFA.

Solution: For the given transition diagram we will first construct the transition table.

State	0	1
→q0	q0	q1

q1	{q1, q2}	q1
*q2	q2	{q1, q2}

Now we will obtain δ' transition for state q0.

1. δ'([q0], 0) = [q0]
2. δ'([q0], 1) = [q1]

The δ' transition for state q1 is obtained as:

1. δ'([q1], 0) = [q1, q2] (**new** state generated)
2. δ'([q1], 1) = [q1]

The δ' transition for state q2 is obtained as:

1. δ'([q2], 0) = [q2]
2. δ'([q2], 1) = [q1, q2]

Now we will obtain δ' transition on [q0],q1, q2].

1. δ'([q1, q2], 0) = δ(q1, 0) ∪δ(q2, δ(q2, 0) = {q1, q2} ∪δ(q2, {q2}
   = [q0],q1, q2]

δ'([q1, q2], 1) = δ(q1, 1) ∪δ(q2, δ(q2, 1)
= {q1} ∪δ(q2, {q1, q2}
= {q1, q2}
= [q0],q1, q2]

The state [q0],q1, q2] is the final state as well because it contains a final state q2. The transition table for the constructed DFA will be:

State	**0**	**1**
→[q0]	[q0],q0]	[q0],q1]
[q0],q1]	[q0],q1, q2]	[q0],q1]
*[q2]	[q0],q2]	[q0],q1, q2]
*[q1, q2]	[q0],q1, q2]	[q0],q1, q2]

The state q2 can be eliminated because q2 is an unreachable state.

Example 2:

Convert the given NFA to DFA.

Solution: For the given transition diagram we will first construct the transition table.

State	0	1
→q0	{q0, q1}	{q1}
*q1	ϕ	{q0, q1}

Now we will obtain δ' transition for state q0.

δ'([q0], 0) = {q0, q1}
= [q0,q0, q1] (**new** state generated)
δ'([q0], 1) = {q1} = [q1]

The δ' transition for state q1 is obtained as:

1. δ'([q1], 0) = ϕ
2. δ'([q1], 1) = [q0, q1]

Now we will obtain δ' transition on [q0,q0, q1].

δ'([q0, q1], 0) = δ(q0, 0) ∪δ(q2, δ(q1, 0)
= {q0, q1} ∪δ(q2, ϕ
= {q0, q1}
= [q0,q0, q1]

Similarly,

δ'([q0, q1], 1) = δ(q0, 1) ∪δ(q2, δ(q1, 1)
= {q1} ∪δ(q2, {q0, q1}
= {q0, q1}
= [q0,q0, q1]

As in the given NFA, q1 is a final state, then in DFA wherever, q1 exists that state becomes a final state. Hence in the DFA, final states are [q0,q1] and [q0,q0, q1]. Therefore set of final states F = {[q1], [q0, q1]}.

The transition table for the constructed DFA will be:

State	0	1
→[q0]	[q0],q0, q1]	[q0],q1]
*[q1]	ϕ	[q0],q0, q1]
*[q0, q1]	[q0],q0, q1]	[q0],q0, q1]

Even we can change the name of the states of DFA.

Suppose

1.A = [q0],q0]
2.B = [q0],q1]
3. C = [q0],q0, q1]

Lec-6

Minimization of DFA

Minimization of DFA means reducing the number of states from given FA. Thus, we get the FSM(finite state machine) with redundant states after minimizing the FSM.

We have to follow the various steps to minimize the DFA. These are as follows:

Step 1: Remove all the states that are unreachable from the initial state via any set of the transition of DFA.

Step 2: Draw the transition table for all pair of states.

Step 3: Now split the transition table into two tables T1 and T2. T1 contains all final states, and T2 contains non-final states.

Step 4: Find similar rows from T1 such that:

1. 1. δ (q, a) = p
2. 2. δ (r, a) = p

That means, find the two states which have the same value of a and b and remove one of them.

Step 5: Repeat step 3 until we find no similar rows available in the transition table T1.

Step 6: Repeat step 3 and step 4 for table T2 also.

Step 7: Now combine the reduced T1 and T2 tables. The combined transition table is the transition table of minimized DFA.

Example:

Solution:

Step 1: In the given DFA, q2 and q4 are the unreachable states so remove them.

Step 2: Draw the transition table for the rest of the states.

State	0	1
→q0	q1	q3
q1	q0	q3
*q3	q5	q5
*q5	q5	q5

Step 3: Now divide rows of transition table into two sets as:

1. One set contains those rows, which start from non-final states:

State	0	1
q0	q1	q3
q1	q0	q3

2. Another set contains those rows, which starts from final states.

State	0	1
q3	q5	q5

q5	q5	q5

Step 4: Set 1 has no similar rows so set 1 will be the same.

Step 5: In set 2, row 1 and row 2 are similar since q3 and q5 transit to the same state on 0 and 1. So skip q5 and then replace q5 by q3 in the rest.

State	**0**	**1**
q3	q3	q3

Step 6: Now combine set 1 and set 2 as:

State	**0**	**1**
→q0	q1	q3
q1	q0	q3
*q3	q3	q3

Now it is the transition table of minimized DFA.

Lec-7

Mealy Moore Machine

Finite automata may have outputs corresponding to each transition. There are two types of finite state machines that generate output −

- Mealy Machine
- Moore machine

Mealy Machine

A Mealy Machine is an FSM whose output depends on the present state as well as the present input.

It can be described by a 6 tuple $(Q, \sum, O, \delta, X, q_0)$ where −

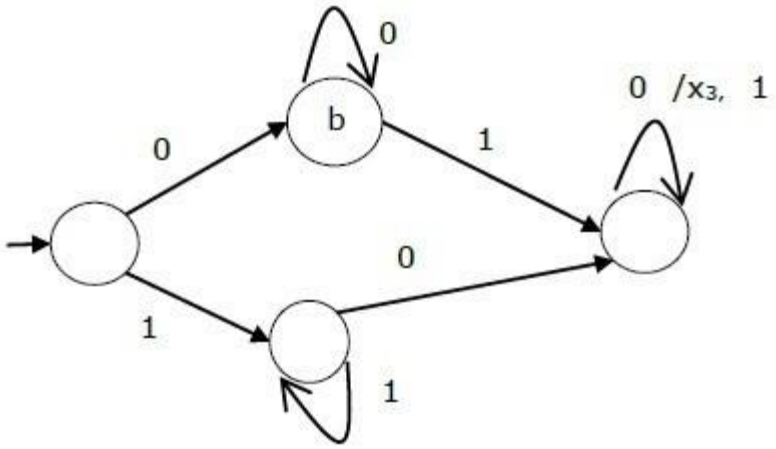
- **Q** is a finite set of states.
- $\sum$ is a finite set of symbols called the input alphabet.
- **O** is a finite set of symbols called the output alphabet.
- **δ** is the input transition function where $\delta: Q \times \sum \rightarrow Q$
- **X** is the output transition function where $X: Q \times \sum \rightarrow O$

- **q_0** is the initial state from where any input is processed ($q_0 \in Q$). Q).

The state table of a Mealy Machine is shown below −

Present state	Next state			
	input = 0		input = 1	
	State	Output	State	Output
→ a	b	x_1	c	x_1
b	b	x_2	d	x_3
c	d	x_3	c	x_1
d	d	x_3	d	x_2

The state diagram of the above Mealy Machine is −



Moore Machine

Moore machine is an FSM whose outputs depend on only the present state.

A Moore machine can be described by a 6 tuple (Q, ∑, O, δ, X, q_0) where −

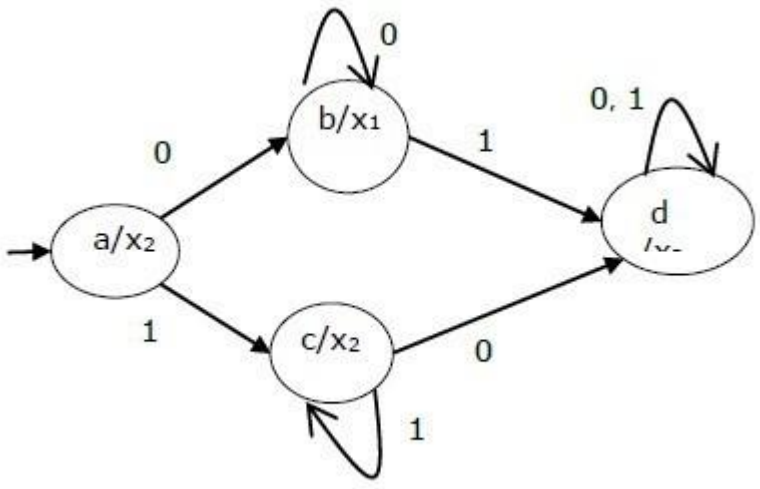
- **Q** is a finite set of states.
- ∑ is a finite set of symbols called the input alphabet.

- **O** is a finite set of symbols called the output alphabet.
- **δ** is the input transition function where δ: Q × ∑ → Q
- **X** is the output transition function where X: Q → O
- **q_0** is the initial state from where any input is processed ($q_0 \in Q$). Q).

The state table of a Moore Machine is shown below −

Present state	Next State		Output
	Input = 0	Input = 1	
→ a	b	c	x_2
b	b	d	x_1
c	c	d	x_2
d	d	d	x_3

The state diagram of the above Moore Machine is −



Mealy Machine vs. Moore Machine

The following table highlights the points that differentiate a Mealy Machine from a Moore Machine.

Mealy Machine	Moore Machine

Output depends both upon the present state and the present input	Output depends only upon the present state.
Generally, it has fewer states than Moore Machine.	Generally, it has more states than Mealy Machine.
The value of the output function is a function of the transitions and the changes, when the input logic on the present state is done.	The value of the output function is a function of the current state and the changes at the clock edges, whenever state changes occur.
Mealy machines react faster to inputs. They generally react in the same clock cycle.	In Moore machines, more logic is required to decode the outputs resulting in more circuit delays. They generally react one clock cycle later.

Moore Machine to Mealy Machine

Algorithm 4

Input − Moore Machine

Output − Mealy Machine

Step 1 − Take a blank Mealy Machine transition table format.

Step 2 − Copy all the Moore Machine transition states into this table format.

Step 3 − Check the present states and their corresponding outputs in the Moore Machine state table; if for a state Q_i output is m, copy it into the output columns of the Mealy Machine state table wherever Q_i appears in the next state.

Example

Let us consider the following Moore machine −

Present State	**Next State**		**Output**
	a = 0	**a = 1**	

→ a	d	b	1
b	a	d	0
c	c	c	0
d	b	a	1

Now we apply Algorithm 4 to convert it to Mealy Machine.

Step 1 & 2 –

Present State	**Next State**			
	a = 0		**a = 1**	
	State	**Output**	**State**	**Output**
→ a	d		b	
b	a		d	
c	c		c	
d	b		a	

Step 3 –

Present State	**Next State**			
	a = 0		**a = 1**	
	State	**Output**	**State**	**Output**

=> a	d	1	b	0
b	a	1	d	1
c	c	0	c	0
d	b	0	a	1

Mealy Machine to Moore Machine

Algorithm 5

Input − Mealy Machine

Output − Moore Machine

Step 1 − Calculate the number of different outputs for each state (Q_i) that are available in the state table of the Mealy machine.

Step 2 − If all the outputs of Qi are same, copy state Q_i. If it has n distinct outputs, break Q_i into n states as Q_{in} where **n** = 0, 1, 2.....

Step 3 − If the output of the initial state is 1, insert a new initial state at the beginning which gives 0 output.

Example

Let us consider the following Mealy Machine −

Present State	**Next State**			
	a = 0		**a = 1**	
	Next State	**Output**	**Next State**	**Output**
→ a	d	0	b	1
b	a	1	d	0

c	c	1	c	0
d	b	0	a	1

Here, states 'a' and 'd' give only 1 and 0 outputs respectively, so we retain states 'a' and 'd'. But states 'b' and 'c' produce different outputs (1 and 0). So, we divide **b** into $\mathbf{b_0}$, $\mathbf{b_1}$ and **c** into $\mathbf{c_0}$, $\mathbf{c_1}$.

Present State	**Next State**		**Output**
	a = 0	**a = 1**	
→ a	d	b_1	1
b_0	a	d	0
b_1	a	d	1
c_0	c_1	C_0	0
c_1	c_1	C_0	1
d	b_0	a	0